

Reviewer Pack — Minimal Rank of Tensor Product Coefficients over Boolean Fun...

Entry b62055de9097 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 22:23:17 UTC

Minimal Rank of Tensor Product Coefficients over Boolean Functions vs DPLL Proof Length

Entry ID: b62055de9097 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 22:23:17 UTC

1. Conjecture

Field A (mathematical branch): Tensor Analysis (Tensor Product Coefficients) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every homogeneous polynomial f in two variables x and y with coefficients in a field F , let $r(f)$ denote the minimal rank of the tensor product coefficient matrix associated with f over the Boolean ring. Then, for all instances of SAT problems, the DPLL proof length $t^*(F)$ is lower-bounded by the logarithm base 2 of $r(f)$.’, ‘Equivalently, if there exists a polynomial-size circuit C such that every satisfying assignment of a CNF F implies that f evaluates to 1 on C , then $r(f) \leq O(|C|^2 \log |F|)$, where $|F|$ is the number of variables in F .’]

Rationale (proposer’s reasoning):

[‘The minimal rank of a tensor product coefficient matrix has been studied in tensor analysis and is closely related to multivariate polynomial properties. By exploring this invariant, we might uncover a novel connection between the structure of polynomials and the complexity of resolution proofs.’, ‘Tensor products are fundamental in linear algebra and have been used to study various problems in mathematics and physics. This conjecture suggests that such algebraic structures could be relevant to understanding proof complexity, particularly for SAT.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 573274207f41bec2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNF instances F with variable count $n \leq 40$, the logarithm base 2 of $r(f)$ (where $r(f)$ is the minimal rank of the tensor product coefficient matrix associated with f over the Boolean ring) is less than or equal to $t(F)$ (*the DPLL*

proof length), AND for at least 80% of the instances, the difference $|\log_2(r(f)) - t(F)| \leq 1$.
The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor product coefficients AND DPLL proof length - SAT problem AND minimal rank tensor analysis Boolean functions - circuit complexity AND polynomial-size circuit resolution proof

Top relevant hits considered: - [s2:10.1109/FOCS.2016.40] How Limited Interaction Hinders Real Communication (and What It Means for Proof and Circuit Complexity) - [s2:2411.14267] Truly Supercritical Trade-Offs for Resolution, Cutting Planes, Monotone Circuits, and Weisfeiler-Leman

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for j in range(n):
        i_max = rank
        for i in range(rank, m):
            if abs(A[i][j]) > abs(A[i_max][j]):
                i_max = i
        if A[i_max][j] == 0:
            continue
        A[rank], A[i_max] = A[i_max], A[rank]
        for i in range(m):
            if i != rank:
                factor = -A[i][j] / A[rank][j]
                for k in range(n):
                    A[i][k] += factor * A[rank][k]
        rank += 1
    return rank
```

```

def minimal_rank(C):
    n = len(C)
    A = [[0] * (n + 1) for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = C[i][j]
        A[i][n] = 1
    return gaussian_elimination(A)

def dpll(CNF, assignment=None):
    if assignment is None:
        assignment = {}
    if not CNF:
        return True
    for clause in CNF:
        if any(lit in assignment and (assignment[lit] == 0) for lit in clause):
            continue
        new_assignment = assignment.copy()
        new_assignment[clause[0]] = 1
        if dpll(CNF, new_assignment):
            return True
        new_assignment[clause[0]] = -1
        if dpll(CNF, new_assignment):
            return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    CNF = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
        CNF.append(clause)

    f = lambda x, y: sum([sum([C[i][j] * x**(i+1) * y**(j+1) for j in range(n)]) for i in range(n)])
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = f(i, j)

    r_f = minimal_rank(C)
    t_F = 1
    while not dpll(CNF, {}):
        t_F += 1

    log_r_f = math.log2(r_f) if r_f > 0 else float('-inf')
    diff = abs(log_r_f - t_F)

    return {
        "metric_name": "log_r_f_minus_t_F",
        "metric_value": diff,
        "instances_tested": 1,
        "conjecture_holds": diff <= 1,
        "counterexample": "" if diff <= 1 else f"Counterexample found with n={n}, log2(r(f))={log_r_f}"
    }

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_diff = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_diff) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 99, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 79, in run_trial
    while not dpll(CNF, {}):
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 56, in dpll
    if dpll(CNF, new_assignment):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 56, in dpll
    if dpll(CNF, new_assignment):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 56, in dpll
    if dpll(CNF, new_assignment):
        ~~~~~
  [Previous line repeated 994 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_07e9068c.py", line 52, in dpll
    if any(lit in assignment and (assignment[lit] == 0) for lit in clause):
        ~~~~~
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be verified. | next: Re-run the test to verify the conjecture under the specified conditions.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13114
2	preregistration	ollama_remote	glm4:latest	0	0	6324
3	novelty	ollama_remote	glm4:latest	0	0	4717
4	novelty	ollama_remote	glm4:latest	0	0	6025
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17321
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15566
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14082
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12602
9	judge	ollama_remote	glm4:latest	0	0	17605

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 107355 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b62055de9097.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b62055de9097.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b62055de9097.tar.gz` (if generated)